Design Architecture

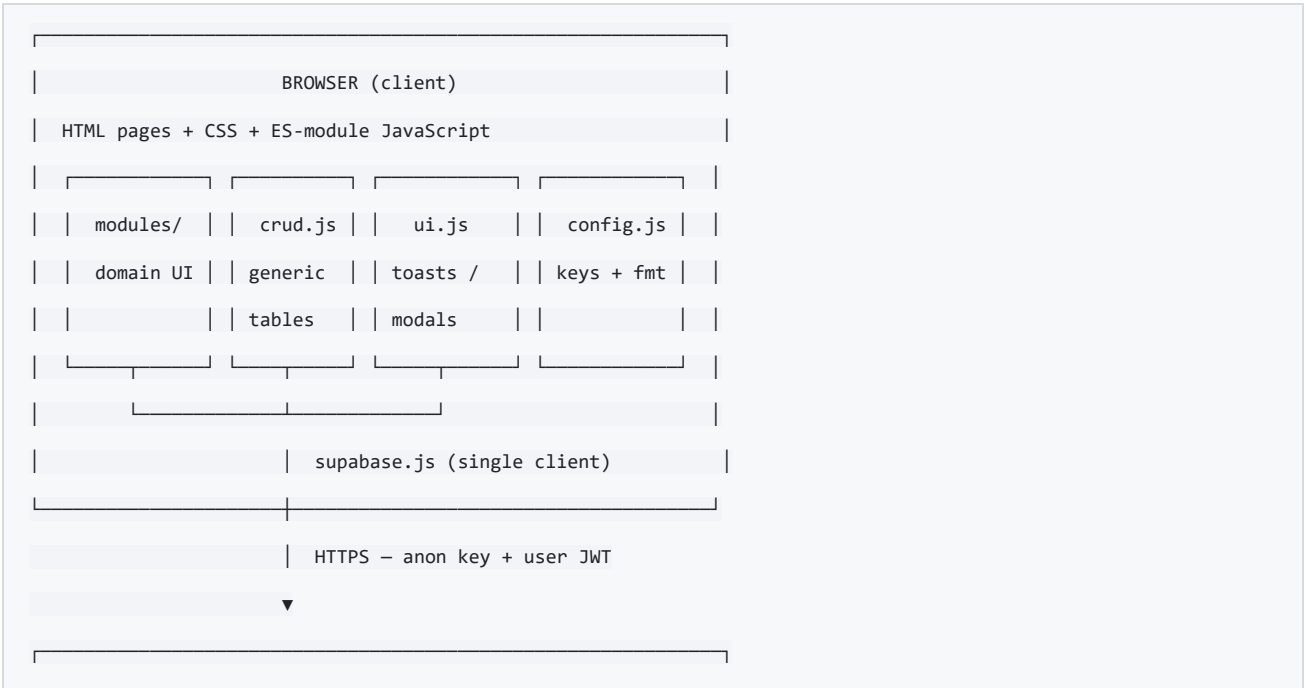
Cake Slice Factory — Sales, Inventory & Expense System

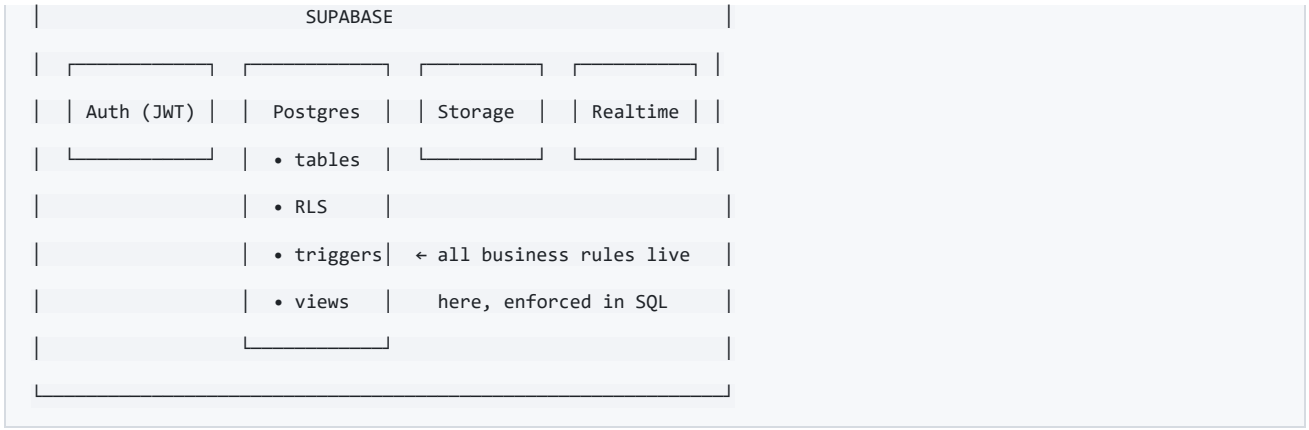
Field	Value
Product	Cake Slice Factory
Version	1.0
Last updated	2026-10-21
Stack	HTML + CSS + JavaScript (no build step) · Supabase (Postgres, Auth, Storage, Realtime)
Companion docs	PRD.md, ../supabase/schema.sql

1. Architecture Overview

1.1 Style

A client–BaaS (Backend-as-a-Service) architecture. There is no custom application server. The browser is a thin, modular presentation layer; Supabase Postgres is the single source of truth and owns all business rules that must not be bypassable (stock math, totals, permissions).





1.2 Architectural principles

The database is the referee. Stock deductions, totals, order numbers and permissions are implemented in SQL, so a buggy or hostile client cannot corrupt data.

No build step. Plain ES modules served statically — easy to host, easy to hand over, no toolchain rot.

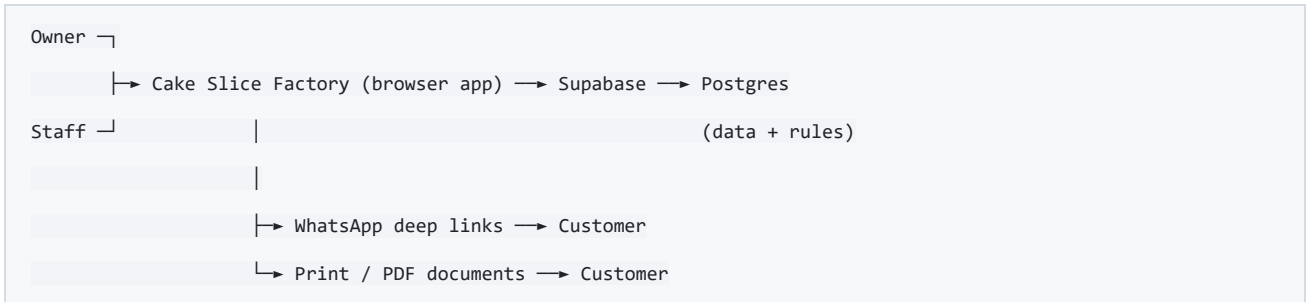
One module per domain. Each screen is a self-contained module with a single render() entry point.

Generic where possible, custom where necessary. Simple entities (vendors, ingredients, products, customers, suppliers) use one reusable CRUD engine; orders, production and reports get bespoke code.

Nothing is truly deleted. Soft deletes + immutable ledger + audit log preserve history.

Idempotent side effects. Completing/reversing an order adjusts stock exactly once.

2. System Context



Actor	Interaction
Owner	Manages catalog, prices, users; reviews reports and audit log
Staff	Records orders, customers, payments, production, expenses

Customer	Receives quote/invoice/receipt (print or WhatsApp); no direct system access in v1
Supabase	Identity provider, database, file storage, change feed

3. Component Architecture (frontend)

```
dashboard.html
└── js/app.js ............................. router + nav + session guard
    ├── js/config.js ...................... Supabase URL/key, currency formatting
    ├── js/supabase.js .................... one client instance, pagination helpers
    ├── js/auth.js ........................ signIn/out, profile, cached role, first-owner claim
    ├── js/theme.js ....................... saved/system light-dark preference + toggle wiring
    ├── js/ui.js .......................... toasts, modals, forms, tables, CSV/Excel/Word/PDF/print
    ├── js/crud.js ........................ generic list+form+delete+export engine
    └── js/modules/
        ├── dashboard.js ....... KPIs, low-stock, due orders
        ├── products.js ........ finished cakes (ready-made + custom)
        ├── ingredients.js ..... ingredient stock
        ├── vendors.js ......... supplier records
        ├── recipes.js ......... bill of materials + cost roll-up
        ├── production.js ...... batches (consumes ingredients, adds stock)
        ├── purchases.js ....... items received (purchase orders + direct receipts)
        ├── orders.js .......... sales, quotes and the order detail/document view
        ├── payments.js ........ payment ledger, refunds and debtors
        ├── documents.js ....... issued quote, invoice and receipt register
        ├── calendar.js ........ month view of pick-ups and deliveries
        ├── deliveries.js ...... dispatch board and rider status
        ├── customers.js ....... customer records + history
        ├── expenses.js ........ expenses + receipts
        ├── reports.js ......... all summaries
        └── admin.js ........... users, settings, audit log, trash (owner)
```

3.1 Module contract

Every module exports a render(root) function. app.js owns routing and passes the container.

```
// js/modules/ingredients.js
```

```
export async function render(root) {
  root.innerHTML = shellHTML();
  await loadRows(root);
}
```

Simple entities do not write this by hand — they declare a spec consumed by crud.js:

```
export const ingredients = defineCrudModule({
  table: 'ingredients',
  title: 'Ingredients',
  ownerOnly: true,
  fields: [
    { name: 'name',         label: 'Ingredient',       type: 'text', required: true },
    { name: 'unit',         label: 'Unit',             type: 'text', default: 'kg' },
    { name: 'cost_per_unit',label: 'Cost / unit',      type: 'number' },
    { name: 'stock_qty',    label: 'Stock',            type: 'number' },
    { name: 'low_stock_at', label: 'Low stock at',     type: 'number' },
    { name: 'vendor_id',    label: 'Vendor',           type: 'select',
      optionsFrom: { table: 'vendors', label: 'name' } },
  ],
  columns: ['name', 'unit', 'stock_qty', 'cost_per_unit'],
});
```

3.2 State management

Deliberately minimal — no framework, no global store.

Server state is fetched on demand and re-fetched after a mutation.

Session/role is cached in auth.js for the lifetime of the page.

Per-module state (current filter, search term, loaded rows) lives in module-local variables.

Lookups (e.g. vendor names) are loaded once per render and memoised in a module-level map.

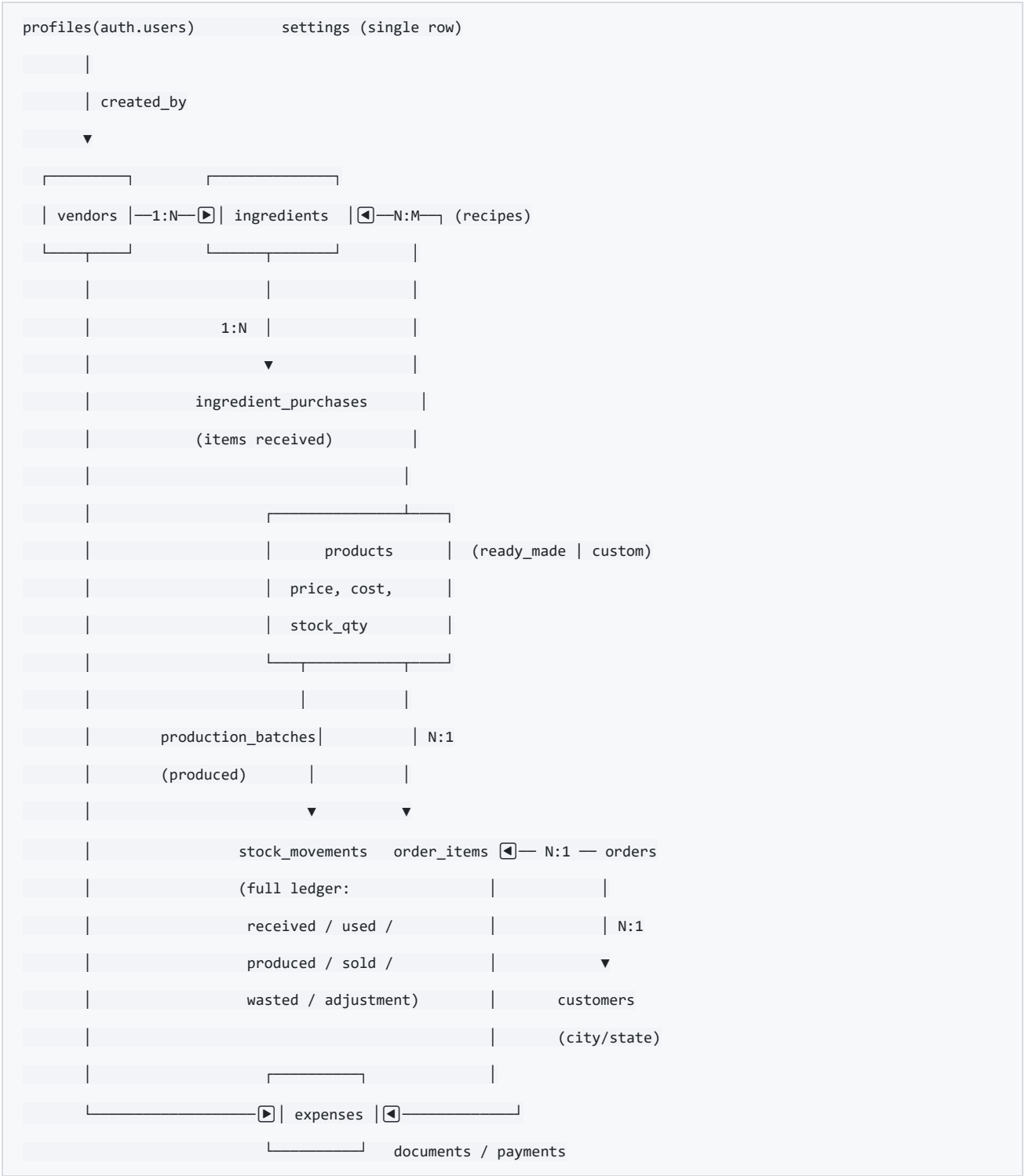
This is appropriate because the data set is small-to-moderate and the app is form-driven, not real-time collaborative. Realtime subscriptions (Phase 6) will invalidate module caches on change.

3.3 Rendering model

Imperative DOM with innerHTML for static structure and event delegation for interaction. All user data interpolated into HTML passes through esc() to prevent injection.

4. Data Architecture

4.1 Entity relationships



4.2 Tables (base — supabase/schema.sql)

Table	Key columns	Notes
settings	business_name, currency, tax_rate, logo_url	Single row (id = true)
profiles	id → auth.users, role owner|staff, active	Created by trigger on sign-up
vendors	name, contact, phone, supplies	Soft-deleted
ingredients	name, unit, cost_per_unit, stock_qty, low_stock_at, vendor_id	Soft-deleted
products	name, sku, kind, price, cost_price, stock_qty, track_stock	Ready-made & custom
recipes	product_id, ingredient_id, qty	Unique per pair
customers	name, phone, channel, city, state	Drives location reports
orders	order_no, customer_id, kind, channel, fulfilment, status, delivery_*, subtotal, discount, delivery_fee, total, amount_paid, stock_deducted	Central sales record
order_items	order_id, product_id, qty, unit_price, line_total (generated)	Line totals computed by Postgres
payments	order_id, amount, method, paid_on	Negative amount = refund
documents	order_id, doc_type, doc_no, snapshot (jsonb)	Quote/Invoice/Receipt
production_batches	batch_no, product_id, qty_produced	Consumes recipe, adds stock
stock_movements	ingredient_id / product_id, change_qty, reason	Immutable ledger
ingredient_purchases	vendor_id, ingredient_id, qty, unit_cost	Items received
expenses	category, vendor_id, order_id, amount, spent_on	All non-ingredient spend

4.3 Tables (additions — supabase/002_core_additions.sql)

Table	Purpose	PRD ref
audit_log	Immutable record of create/update/delete on money tables	A2, M11-FR-02
order_status_history	Timestamped status transitions with actor	M6-FR-08
purchase_orders + purchase_order_items	Order ahead from vendors; receiving creates stock	B6, M4-FR-04
stock_takes + stock_take_items	Physical count vs system; posts variance adjustments	B7, M5/M3
unit_conversions	Buy in bags, consume in kg/g	B4, M3-FR-04
attachments	Cake references & expense receipts (Supabase Storage paths)	B10, M9-FR-03
notifications	Low stock, orders due, overdue balances	B12, M11-FR-04
price_history	Every selling-price change with actor	B11, M2-FR-03

4.4 Reporting views (security_invoker = true)

Views run with the caller's permissions, so RLS still applies.

View	Answers
v_stock_alerts	What is low in stock (products + ingredients)?
v_product_sales	Which product sold the most?
v_top_customers	Which customer bought the most?
v_top_locations	Which city/state earns the most?
v_ingredient_restock	Which ingredients were restocked most, and how much?
v_expense_summary	Expenses by month and category
v_sales_trend	Orders and revenue per day

5. Security Architecture

5.1 Authentication

Supabase Auth (email + password, JWT). The client holds only the anon key; the

service-role key never reaches the browser.

The JWT identifies the user; profiles.role grants authorisation.

Sessions persist across reloads and auto-refresh.

The first signed-in account can call `public.claim_owner()`. The function takes a transaction-level advisory lock, confirms that no owner exists, marks the caller as owner, and sets `csf.owner_bootstrap` so the role guard allows that one checked transition. Later calls fail. All other role changes remain owner-only.

5.2 Authorisation — Row Level Security

RLS is enabled on every table. Two SQL helper functions encapsulate the rules:

```
public.my_role()      -- app_role for auth.uid()
public.is_owner()     -- role = 'owner'
public.can_operate()  -- role in ('owner','staff')
```

Policy shape applied per table class:

Table class	SELECT	INSERT/UPDATE/DELETE
settings, profiles	authenticated / own-or-owner	owner
vendors, ingredients, products, recipes	authenticated	owner only
customers, orders, order_items, payments, documents, production_batches, stock_movements, ingredient_purchases, expenses	authenticated	owner or staff

```
Request ──► PostgREST ──► RLS policy ──► rows
                              │
                              ├─ staff writing to products  ──► 0 rows (blocked)
                              └─ staff creating an order    ──► allowed
```

5.3 Threat model & mitigations

Threat	Mitigation
Client tampering to raise privileges	All authorisation in RLS; UI hiding is cosmetic only
Leaked anon key	Anon key is public by design; RLS protects data
Injection via form input	Parameterised PostgREST queries; esc() before innerHTML
Accidental/malicious deletion	Soft delete + owner-only delete + audit log
Silent data loss on flaky network	Write confirmation + retry queue (Phase 6)
Customer phone exposure	Authenticated-only read; no public endpoints

6. Business Logic Placement

Business rules run in Postgres triggers, not in JavaScript. Rationale: they must hold for every client, remain correct if code is refactored, and execute atomically with the data change.

Trigger	Fires on	Effect	Rule
handle_new_user	auth.users insert	Create profile (staff)	M1-FR-03
touch_updated_at	update on main tables	Maintain updated_at	—
set_order_no / set_doc_no / set_batch_no	insert	Generate ORD-/QUO-/INV-/RCT-/BAT- numbers	BR-16
on_ingredient_purchase	ingredient_purchases insert	Ingredient stock ↑, ledger row, cost update	M4-FR-02, BR-18
on_production_batch	production_batches insert	Recipe ingredients ↓, product stock ↑, ledger rows	M5-FR-02/03
recalc_order_totals	order_items insert/update/delete	Recompute subtotal	BR-14
recalc_order_total_only	orders insert/update	total = subtotal − discount + fee + tax	BR-14

sync_amount_paid	payments insert/update/delete	Recompute amount_paid	BR-15
on_order_completed	orders update → completed	Deduct stock once (stock_deducted guard)	BR-04
reverse_order_stock	orders update → cancelled/refunded	Return stock once	BR-05
audit_row_change	insert/update/delete on money tables	Write audit_log entry	A2

6.1 Stock conservation invariant

Every quantity in the system is explained by the ledger:

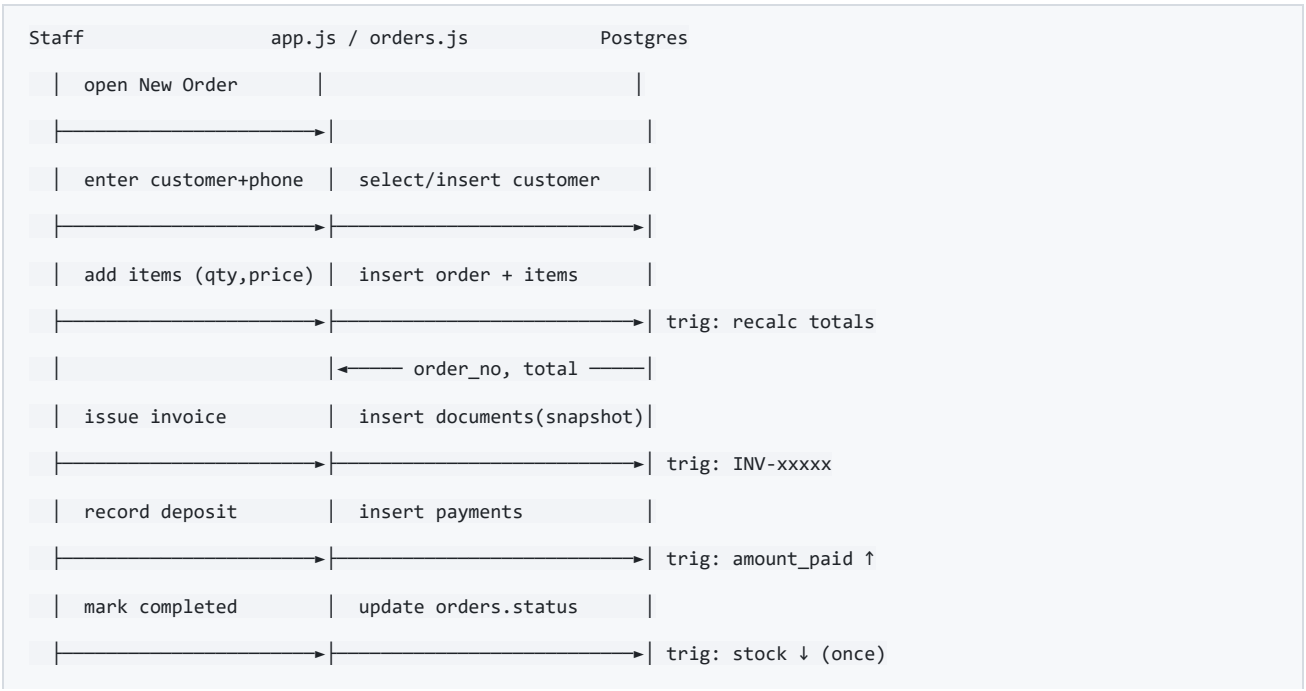
```
ingredient.stock_qty  ≡  Σ stock_movements.change_qty      (per ingredient)

product.stock_qty     ≡  Σ stock_movements.change_qty      (per product)
```

Any stock take variance is itself written as an adjustment movement, so the invariant holds. A monthly reconciliation query can prove it — this is the correctness guarantee of the design.

7. Key Flows

7.1 Record and complete an order



```
  |◄── receipt printable ──|                          |
```

7.2 Production batch

```
Baker records batch(product, qty)
        |
        ▼
insert production_batches ──► trigger
        ├─ for each recipe line:  ingredients.stock_qty -= qty × batch_qty
        |                         stock_movements(reason='used_in_production')
        ├─ products.stock_qty += batch_qty
        └─ stock_movements(reason='produced')
   (atomic – all or nothing)
```

7.3 Items received

```
Purchase recorded (vendor, ingredient, qty, unit_cost)
        |
        ▼
insert ingredient_purchases ──► trigger
        ├─ ingredients.stock_qty += qty
        ├─ ingredients.cost_per_unit = latest non-zero unit cost
        └─ stock_movements(reason='purchase_received')
```

7.4 Cancel / refund a completed order

```
orders.status: completed → cancelled
        |
        ▼
trigger reverse_order_stock
        ├─ guard: only if stock_deducted = true  (idempotent)
        ├─ products.stock_qty += qty  for each line
        ├─ stock_movements(reason='returned')
        └─ stock_deducted = false
```

8. Error Handling & Resilience

Concern	Approach
Failed write	Every mutation checks the Supabase error; on failure a toast shows the message and the row is not removed from the UI
Duplicate submit	Buttons disabled during await; unique constraints (order_no, sku) as a backstop
Empty state	Each module renders a friendly empty state with a call to action
Loading state	Tables render a skeleton/spinner while fetching
Partial failure in a multi-step flow	Order + items written via a single Postgres function (create_order(...)) so it is atomic
Network loss (Phase 6)	Outbox queue in localStorage; replay on online event; visible "pending sync" badge
Expired session	Any 401 redirects to login, preserving the intended route

9. Directory Structure

```
cake-inventory/
├── README.md
├── docs/
│   ├── PRD.md
│   └── ARCHITECTURE.md
├── index.html                  login / sign-up / first-owner setup
├── dashboard.html              app shell
├── css/
│   └── style.css               light/dark themes, responsive shell, print
├── js/
│   ├── config.js
│   ├── supabase.js
│   ├── auth.js
│   ├── theme.js
│   ├── ui.js
│   ├── crud.js
│   ├── app.js
│   └── modules/
```

```
│       ├── dashboard.js
│       ├── products.js
│       ├── ingredients.js
│       ├── vendors.js
│       ├── recipes.js
│       ├── production.js
│       ├── purchases.js
│       ├── orders.js
│       ├── payments.js
│       ├── documents.js
│       ├── calendar.js
│       ├── deliveries.js
│       ├── customers.js
│       ├── expenses.js
│       ├── reports.js
│       └── admin.js
└── supabase/
    ├── schema.sql              001 — base
    ├── 002_core_additions.sql  002 — controls & safety features
    └── 003_owner_setup.sql     003 — first-run owner claim
```

10. Configuration & Environments

Item	Approach
Supabase URL + anon key	js/config.js (public by design)
Business settings	settings row in the database, editable by the owner in the app
Environments	Two Supabase projects: cakeslice-dev and cakeslice-prod; same schema
Secrets	None in the client. Service-role key is never shipped.
Schema changes	Versioned SQL files, applied in order; every migration is idempotent

11. Performance Considerations

Area	Decision
Payload size	Select only needed columns; use count: 'exact', head: true for KPI numbers
Lists	Server-side pagination (default 25–50 rows) via range()
Lookups	Fetch once per page render and memoise; avoid N+1 queries in table rendering
Reports	Pre-aggregated SQL views so the client fetches small result sets
Assets	Single small CSS file; no framework; modules loaded natively by the browser
Caching	Browser HTTP cache for static assets; Supabase connection reuse via one client

12. Deployment & Operations

Hosting: any static host (Netlify, Vercel, GitHub Pages, or npx serve . locally).

Because there is no build step, deployment is a file copy.

Database: Supabase managed Postgres; schema applied through the SQL editor or Supabase CLI.

Backups: Supabase automated daily backups (depending on plan); plus a full JSON snapshot and CSV/Excel/Word/PDF/print exports the owner can run.

Restore: documented procedure — restore DB snapshot, redeploy static files, re-verify the stock-conservation invariant query.

Monitoring: Supabase logs for API/database errors; client-side error logging with a stack trace and user context.

13. Testing Strategy

Layer	Approach
Database rules (highest value)	SQL test script asserting triggers: totals, stock conservation, idempotent completion, reversal, audit rows
Permissions	Attempt writes as staff and assert they are rejected by RLS
View correctness	Compare each report view to hand-computed expected values on test fixtures

Frontend modules	Manual scripted test pass per module (create/edit/delete/validate/empty state)
End-to-end	Golden path: sign in → add product → add ingredient → recipe → produce → order → pay → complete → invoice → report
Regression	Re-run the SQL invariant suite after every schema migration

14. Architecture Decision Records (ADRs)

#	Decision	Rationale	Trade-off
ADR-01	Supabase over a custom backend	Auth, DB, storage, realtime and RLS out of the box; no server to run	Vendor dependency
ADR-02	No build step (vanilla ES modules)	Zero toolchain, trivially hostable, easy handover	No JSX/TS; larger files; manual asset hashing
ADR-03	Business rules in Postgres triggers	Enforced for every client; atomic; refactor-proof	Logic in SQL, harder to unit test than JS
ADR-04	RLS instead of an API layer	Authorisation cannot be bypassed; less code	Policy changes require SQL migrations
ADR-05	Soft delete everywhere	History and reports never lose data	Queries must filter deleted_at is null
ADR-06	Immutable stock ledger	Provable stock accuracy; supports auditing	Extra writes per operation
ADR-07	Reporting via SQL views	Fast, small payloads, consistent definitions	Schema change needed for new metrics
ADR-08	Custom cakes as order-level data	Made-to-order items have no stock or recipe	Two order shapes to handle
ADR-09	Generic CRUD engine	Fewer lines, consistent UX across simple entities	Less flexible for unusual screens
ADR-10	Money as numeric(14,2)	Exact decimal arithmetic, no float drift	Must format for display

15. Roadmap Alignment

Phase	Architecture work
R0 Foundation	Schema + RLS + auth + shell + CRUD engine
R1 Inventory	Products, ingredients, vendors, recipes modules
R2 Sales	Orders, customers, payments, documents
R3 Production & Expenses	Batch and expense modules; purchase receipts
R4 Reporting & Controls	Report views wired up; audit log, trash, notifications
R5 Hardening	Offline outbox, realtime subscriptions, Storage uploads, PDF export, stock takes, purchase orders
R6 Scale	Multi-branch, forecasting, loyalty